

FIAP | DSA

Aula 3: Fixação de Recursividade Aplicada

Problemas práticos, pilha de chamadas e análise Big-O

Prof. Dr. Erick Toshio Yamamoto

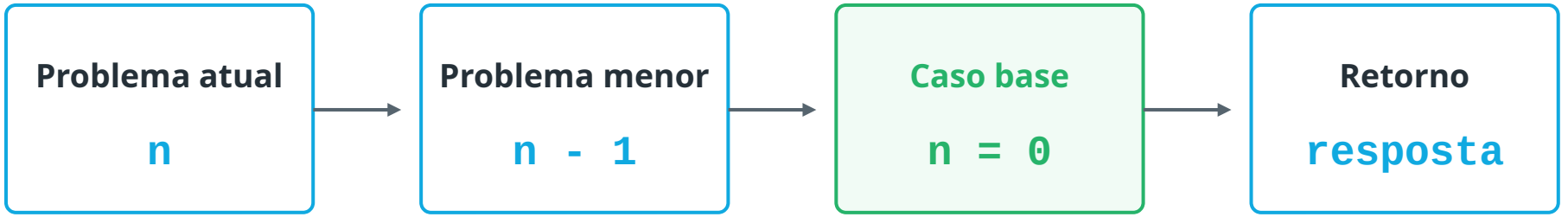
2026

Objetivos da aula

1. Reconhecer caso base, chamada recursiva e redução do problema.
2. Visualizar a pilha de chamadas e a ordem de retorno.
3. Aplicar recursão em números, vetores, strings e busca.
4. Relacionar cada solução recursiva com sua complexidade Big-O.
5. Decidir quando recursividade é adequada e quando uma solução iterativa é melhor.

Modelo mental: o que é recursividade?

Resolver um problema usando uma versão menor dele mesmo.



Problema → menor → menor → ... → caso base → retorno

Pergunta 1

A chamada recursiva aproxima a execução do caso base?

Pergunta 2

Existe uma condição clara que encerra a recursão?

Anatomia de uma função recursiva

```
void contagem(int n) {

    if (n == 0) {      // caso base
        return;
    }

    printf("%d\n", n);

    contagem(n - 1);   // chamada recursiva
}
```

1. Caso base

Condição que encerra a sequência de chamadas.

2. Redução do problema

A cada chamada, n diminui: n → n−1 → n−2...

3. Chamada recursiva

A função chama a si mesma com uma entrada menor.

Sem caso base → risco de Stack Overflow

Exercício 1 — Contagem regressiva

Como imprimir de 5 até 1 sem usar for ou while?

```c
void contagem(int n) {
    if (n == 0) {
        printf("Fim!\n");
        return;
    }

    printf("%d\n", n);
    contagem(n - 1);
}
```

Caso base

n == 0

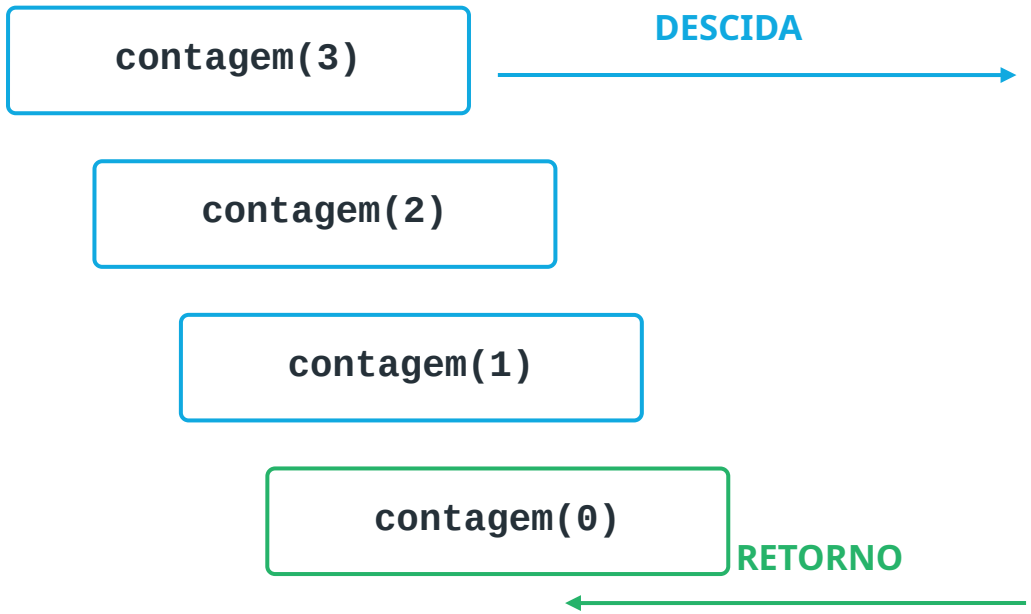
Redução

n → n − 1

Complexidade

Uma chamada por valor de n → O(n)

Pilha de chamadas: contagem(3)



As chamadas ficam aguardando na pilha até atingir o caso base.

Depois do caso base, as chamadas terminam na ordem inversa.

Pilha máxima: O(n) chamadas ativas

Antes ou depois da chamada?

A posição do comando altera a ordem de execução.

Imprime antes da chamada

```
printf("%d ", n);
contagem(n - 1);
```

Para n = 3:

3 2 1

Imprime depois da chamada

```
contagem(n - 1);
printf("%d ", n);
```

Para n = 3:

1 2 3

Antes = descida | Depois = retorno

Exercício 2 — Somatório recursivo

Calcule 1 + 2 + 3 + … + n sem usar laços.

```
int somatorio(int n) {
    if (n == 0)
        return 0;

    return n + somatorio(n - 1);
}
```

Definição

$S(n) = n + S(n-1)$

Caso base

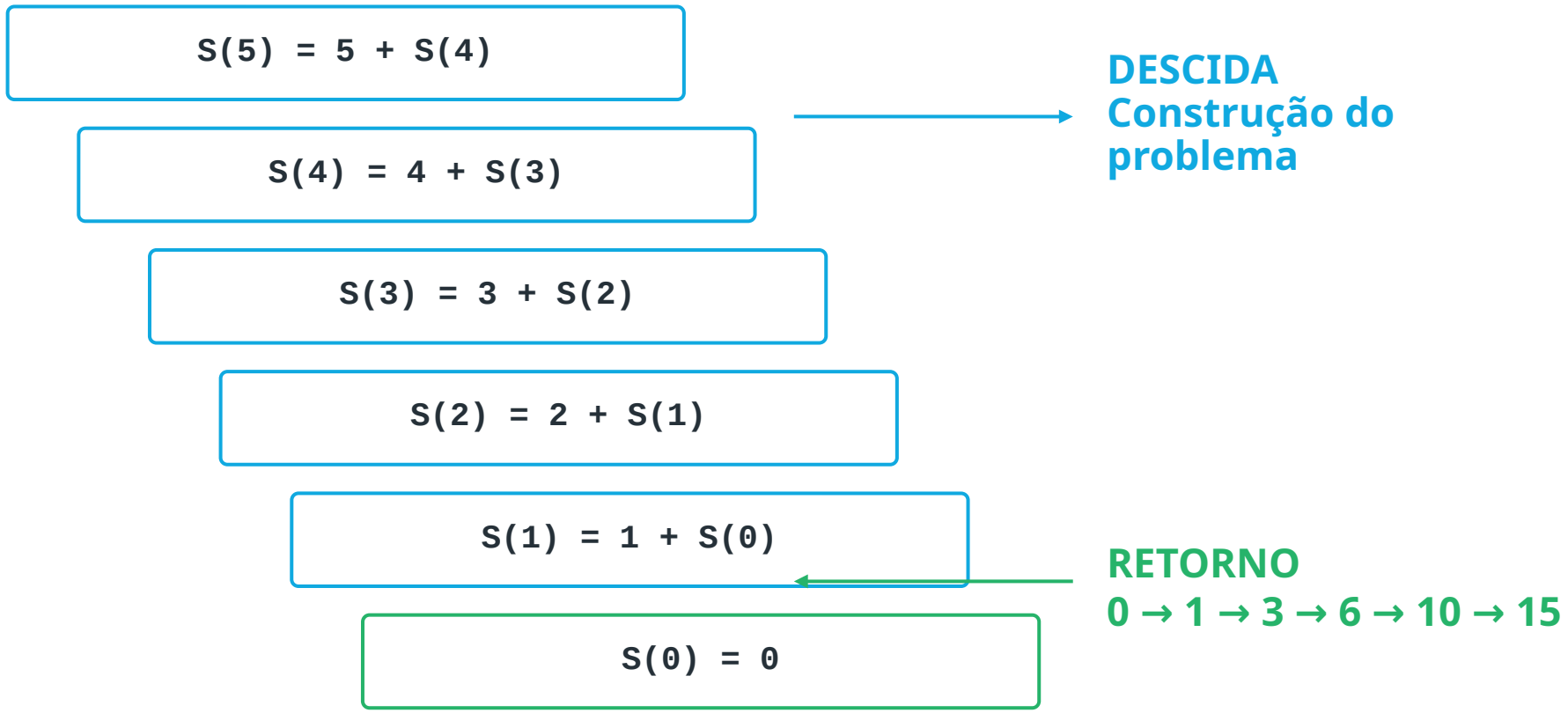
$S(0) = 0$

Complexidade

$T(n) = T(n-1) + c \Rightarrow O(n)$

$S(5) = 5 + 4 + 3 + 2 + 1 = 15$

Rastreando o somatório



Exercício 3 — Potência recursiva

Transformando uma regra matemática em algoritmo.

```
long long potencia(int base, int exp) {
    if (exp == 0)
        return 1;

    return base * potencia(base, exp -
1);
}
```

Relação matemática

$a^n = a \cdot a^{n-1}$

Caso base

$a^0 = 1$

Análise

Uma chamada por expoente → O(n)

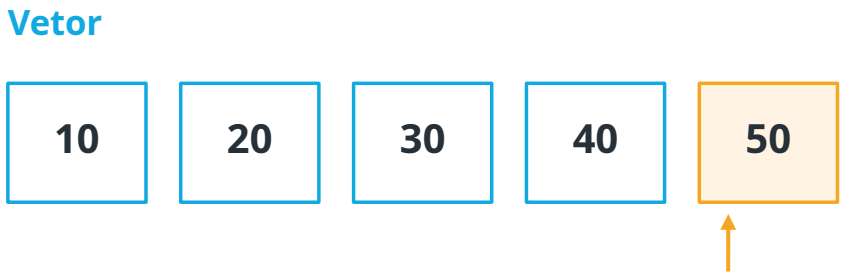
potencia(2, 5) = 32

Exercício 4 — Soma de um vetor

Aplicando recursividade em uma estrutura linear.

```
int somaVetor(int v[], int n) {
    if (n == 0)
        return 0;

    return v[n - 1] + somaVetor(v, n - 1);
}
```



A cada chamada,
n diminui em 1.

Complexidade: O(n)

Exercício 5 — Maior elemento do vetor

```
int maior(int v[], int n) {
    if (n == 1)
        return v[0];

    int anterior = maior(v, n - 1);

    if (v[n - 1] > anterior)
        return v[n - 1];

    return anterior;
}
```

Ideia

“Qual é o maior dos primeiros n−1 elementos?”

Depois

Compare o resultado com v[n−1].

Complexidade

Cada elemento participa de uma comparação → O(n)

Exercício 6 — Inverter uma string

```
void inverter(char texto[], int i) {
    if (i < 0)
        return;

    printf("%c", texto[i]);
    inverter(texto, i - 1);
}
```

FIAP

↓

PAIF

O(n)

Uma chamada por caractere.

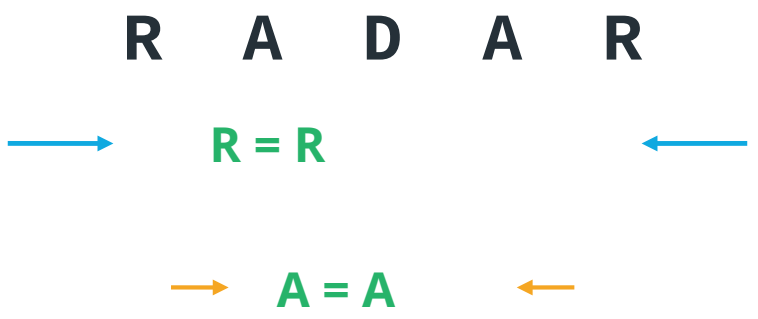
Exercício 7 — Palíndromo

Compare as extremidades e avance para o centro.

```
int palindromo(char t[], int ini, int fim) {
    if (ini >= fim)
        return 1;

    if (t[ini] != t[fim])
        return 0;

    return palindromo(t, ini + 1, fim - 1);
}
```



Cada chamada reduz duas posições → O(n)

Exercício 8 — Busca binária recursiva

O problema é reduzido pela metade a cada chamada.

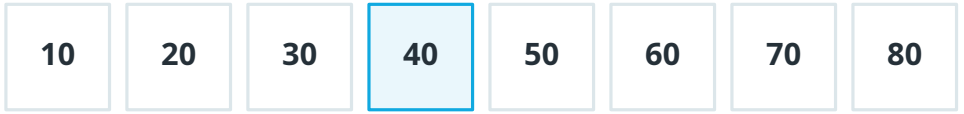
```
int busca(int v[], int ini, int fim, int x) {
    if (ini > fim) return -1;

    int meio = ini + (fim - ini) / 2;

    if (v[meio] == x) return meio;

    if (x < v[meio])
        return busca(v, ini, meio - 1, x);

    return busca(v, meio + 1, fim, x);
}
```



Buscar 70

8 → 4 → 2 → 1

O(log n)

Recursividade ≠ O(n).
A complexidade depende de como o problema é reduzido.

Busca linear × busca binária

Busca linear

Redução:
$n \rightarrow n-1 \rightarrow n-2 \rightarrow \ldots$

Pior caso:
$O(n)$

Busca binária

Redução:
$n \rightarrow n/2 \rightarrow n/4 \rightarrow \ldots$

Pior caso:
$O(\log n)$

A forma de reduzir o problema determina o crescimento

Desafio — Fibonacci recursivo

Uma implementação elegante pode ser computacionalmente cara.

```
long long fibonacci(int n) {
    if (n <= 1)
        return n;

    return fibonacci(n - 1)
         + fibonacci(n - 2);
}
```

Definição

$F(n) = F(n-1) + F(n-2)$

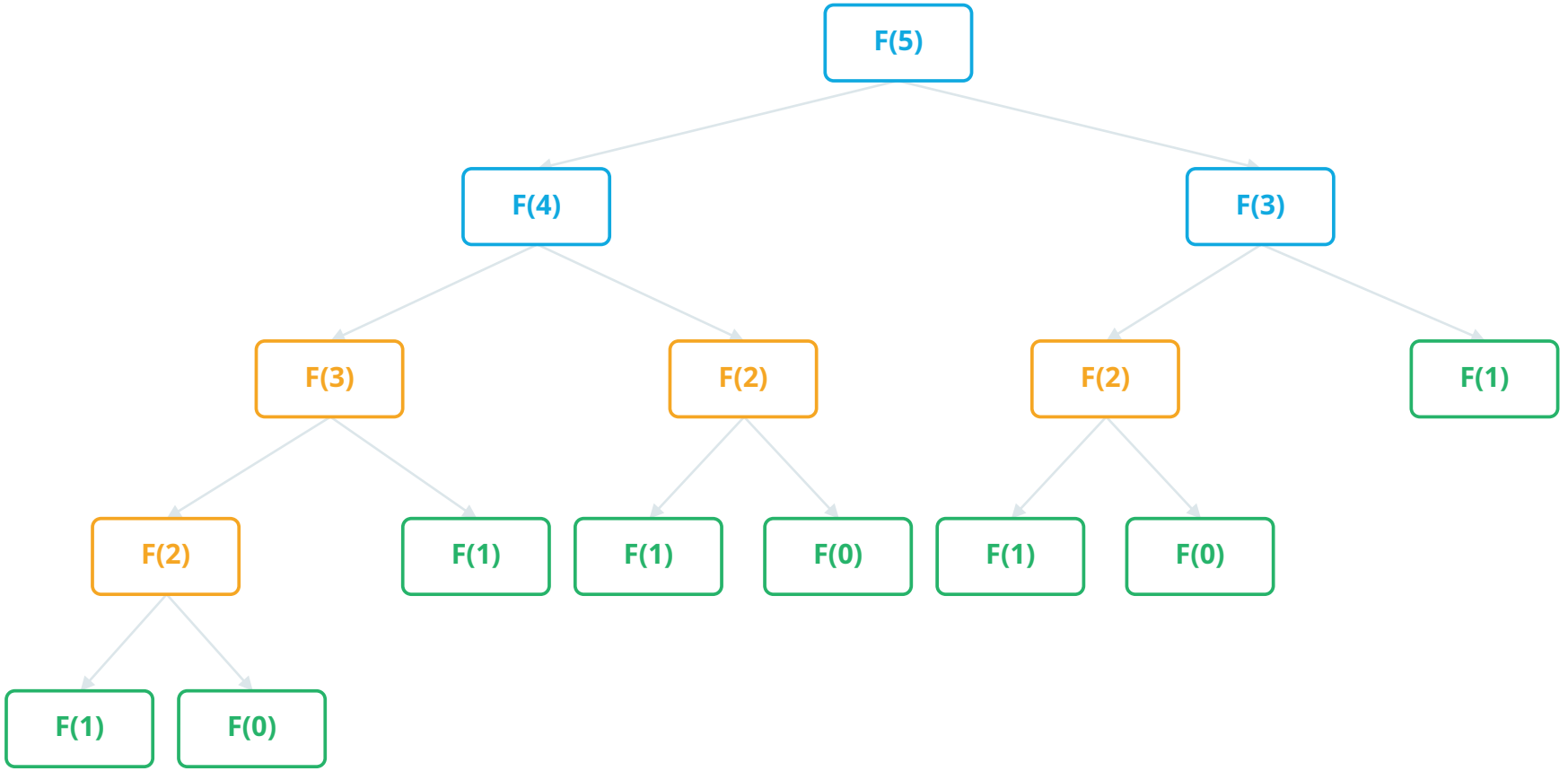
Problema

As mesmas chamadas são recalculadas várias vezes.

Complexidade

Limite superior didático: $O(2^n)$

Árvore de chamadas — Fibonacci(5)



Observe

F(3) aparece 2×

F(2) aparece 3×

F(1) aparece 5×

Há trabalho repetido.

15 chamadas para F(5)

Fibonacci: recursivo × iterativo

Critério	Recursivo ingênuo	Iterativo
Tempo	$O(2^n)$	O(n)
Memória	O(n)	O(1)
Código	Muito próximo da definição	Mais eficiente
Repetição	Recalcula subproblemas	Calcula cada termo uma vez

Recursão elegante ≠ recursão eficiente

Desafio rápido — Qual será a saída?

```
void misterio(int n) {
    if (n <= 0)
        return;

    printf("%d ", n);
    misterio(n - 2);
    printf("%d ", n);
}

// chamada:
misterio(5);
```

1. Descida

5 → 3 → 1

2. Retorno

1 → 3 → 5

Saída: 5 3 1 1 3 5

Use este exercício para verificar se a turma entendeu a diferença entre “antes” e “depois” da chamada recursiva.

Aplicação prática — Navegação em diretórios

Recursão aparece naturalmente em estruturas hierárquicas.




```
void visitar(Pasta pasta) {
    mostrarArquivos(pasta);

    for (cada subpasta) {
        visitar(subpasta);
    }
}
```

Padrão: nó atual + subestruturas

Hands-on — Fixação em grupos

Escolha um problema e implemente sem usar laços na parte recursiva.

Desafio A

Somar apenas os números pares de um vetor.

Desafio B

Contar quantas vezes um caractere aparece em uma string.

Desafio C

Encontrar o menor elemento de um vetor.

Desafio D

Calcular o MDC usando o algoritmo de Euclides recursivo.

Para cada solução: caso base + redução + Big-O

Recursividade e Big-O — revisão

Problema	Complexidade
Contagem regressiva	O(n)
Somatório / vetor	O(n)
Palíndromo	O(n)
Busca binária	O(log n)
Fibonacci ingênuo	$O(2^n)$

Recursividade é uma técnica de implementação — não uma classe de complexidade.

Erros comuns em recursão

Sem caso base

A função nunca encontra uma condição de parada.

Não reduz o problema

A entrada permanece igual ou se afasta do caso base.

Ignorar a pilha

Muitas chamadas ativas consomem memória.

Recursão desnecessária

Um laço simples pode ser mais claro e eficiente.

Sempre pergunte: “essa chamada está mais perto do caso base?”

Checklist antes de escrever uma função recursiva

1. Qual é o caso base?
2. Como o problema fica menor?
3. O retorno está correto?
4. Qual é a profundidade da pilha?
5. Qual é a complexidade?
6. Uma solução iterativa seria mais simples?

Recursão bem definida = problema menor até o caso base